

ARCHITECTURE BRIEF

Agent-Native Apps

Software designed for the user's own agent.

The opportunity is not another chatbot. It is software that becomes useful to an external agent.

Cursor

Codex

MCP

Browser

Memory

Build for the user's agent, not just the user.

[image]

The thesis

The workflow changes when the agent owns the loop

What changes	What to build
• The user brings the agent. The app becomes a surface, not a silo. • The same workspace must support human review and agent action. • The best product is the one the user's agent can actually finish in.	• Local context files, memory, and assets the agent can re-read. • Tool paths that let the agent act through the app, not around it. • A visible shared surface where the user can watch and correct.
Why this matters	Design rule
• If the agent cannot reach the work, the product is only a chatbot. • If the agent cannot see the same state, collaboration breaks down.	• Start with the shared surface and the tool surface together. • Treat context as a product asset, not an implementation detail.

- Context
- Surface
- Tools

The architecture is a loop, not a router.

[image]

Model inversion

App with an agent vs. agent with an app

Old model	New model
• The app embeds a chatbot inside a closed workspace. • Context stops at the app boundary, so the model can assist but not finish. • The user still does the hard part outside the product.	• The user's agent owns the context and brings it into the app. • The app becomes one surface among many in a larger workflow. • The user and the agent can act on the same object at the same time.
What changes	Result
• Moat shifts to context, tools, workflow design, and memory. • The app must be readable by agents from outside the app.	• The app becomes a workspace, not a silo. • The model is useful because it can operate on live state.

App AI

BYO agent

Old model: the app contains the agent. New model: the agent contains the app surface.

[image]

Why now

Three shifts make the category viable today

User agents are real • People are already working inside Cursor, Codex, and similar hosts. • OpenAI's Codex CLI can read, change, and run code locally. • Codex's in-app browser gives a shared view of rendered pages.	Apps expose tools • MCP, CLI, API, and browser access make external action practical. • Notion MCP can read and write workspace content with permissions. • Codex can connect to MCP servers from the CLI or IDE extension.
Context is portable • Files, docs, assets, and memory can live outside one product. • The agent can carry that context across multiple surfaces.	Evidence • OpenAI Codex CLI, Codex app browser, and Notion MCP show the workflow is already here. • The category is not theoretical; the tooling already exists.

- Codex CLI
- Notion MCP
- Browser

The opportunity opens when agents can bring context into an app and act through tools.

[image]

What makes something agent-native

The product needs four ingredients before it is truly BYO-agent friendly

Required ingredients	What is missing without them
• Context: local files, notes, assets, and reusable memory. • Tools: MCP, CLI, API, or browser access that the agent can call. • Surface: a shared workspace both human and agent can edit.	• Without context, the agent repeats instructions. • Without tools, the agent can only suggest work. • Without a shared surface, the work stays invisible.
Design outcome	Definition
• The agent can act, the user can observe, and both can correct. • The system becomes better because the workspace remembers.	• Agent-native software is software designed for the user's agent. • The app is the surface, not the intelligence container.

- Context
- Tools
- Surface
- Memory

Agent-native means shared context + tool access + visible collaboration + compounding memory.

[image]

Reference architecture

Context → agent loop → shared surface → outcome → memory

Context layer • Local files, brand rules, task notes, and project assets. • This is the durable knowledge the agent should re-use. • It makes the product consistent across runs.	Execution layer • Cursor, Codex, or another host runs the agent loop. • The agent calls tools instead of hand-waving the work. • The loop can check, correct, and continue.
Shared surface • The canvas, document, sheet, or browser where work is visible. • This is where humans can review and intervene.	Memory / write-back • The output becomes context for the next turn. • That write-back is what lets quality compound.



Read left to right: context feeds the agent, tools move it into the app, and write-back turns output into future context.

[image]

Context layer

The system improves because the agent can re-read what it already learned

What belongs here	Why it matters
• Markdown notes, decision logs, brand rules, and design specs. • Assets, examples, and the history of what was approved. • Any project truth the agent should not re-discover.	• Context removes repetitive prompting and keeps output consistent. • It compounds value over time instead of resetting every session.
Operational rule	Result
• Write reusable knowledge into files the agent can read. • Treat those files as product surface area.	• The agent gets smarter without shipping code. • The user sees the benefit in fewer corrections.

- Files
- Notes
- Assets
- Memory

Context is the moat because it compounds across tasks and sessions.

[image]

Tool layer

MCP, CLI, API, and browser access are the unlock

Tool paths • MCP for structured integrations across clients. • CLI for local actions, scripts, and developer workflows. • API or browser access when the app surface itself is the workspace.	What parity means • If a user can do it, the agent needs a path to do it too. • The action should succeed or fail visibly and immediately.
Design rule • Expose atomic primitives, not one giant workflow box. • Keep domain logic in the agent loop, not hidden in the tool.	Why it matters • Tool access is a product decision, not just plumbing. • Parity is what makes the app actually usable by an external agent.

- MCP
- CLI
- API
- Browser

The best tool surfaces make actions explicit, composable, and auditable.

[image]

Shared surface

The canvas, doc, sheet, or browser is where collaboration becomes legible

Good surfaces	Design rules
• Canvas for visual work. • Docs for narrative and planning work. • Sheets for structured and tabular work.	• The user and agent must edit the same object. • The state must be visible, reversible, and reviewable.
What to expose	Why it matters
• History, approval, comments, and change tracking. • The current state and the last meaningful action.	• A shared surface lets the user watch the agent work. • It turns AI from a hidden process into a collaboration layer.

- Canvas

Docs

Sheets

Browser

The surface is the product; the agent only makes it useful.

[image]

Human in the loop

The collaboration loop should be part of the product, not a side effect

Watch • The user can see each change live. • Progress should be visible, not hidden. • The agent should show enough state to be trusted.	Correct • Feedback should be cheap and immediate. • Corrections become new context for the next run.
Compounding • The loop teaches the system what good looks like. • A better feedback loop makes the product better over time.	Why it matters • Automation without visibility is fragile. • Collaboration needs review and correction built in.

- Watch
- Correct
- Compound

Visible feedback is what turns automation into collaboration.

[image]

Where the opportunity is

The strongest surfaces already carry context and already host repeated work

Best-fit surfaces	Why they win
• Canvas tools like Miro because the work is already collaborative. • Docs and knowledge bases because context already lives there. • Sheets and suites because structured data is already the workflow.	• They already have a shared object to work on. • They already have users who need reviewable state.
Opportunity logic	Priority signal
• The best wedge is where the app can hold context and expose actions. • Large incumbents still have room because they already own the workflow.	• Look for repeated, visible, auditable work. • That is where agent-native software has room to matter.

- Miro
- Notion
- Sheets
- Adobe

The surface is the product; the agent just amplifies what is already there.

[image]

Design tools

Cursor + Magic Path shows the pattern in practice

How it works	Why it matters
• Open the design app inside the agent host. • Reference local brand files and the design spec. • Let the agent create or revise the page while you watch.	• The design stays consistent because the agent sees the local context. • The result can be pulled straight back into the codebase.
What this proves	Practical benefit
• The app does not need its own embedded agent to be useful. • The app only needs to be an excellent surface for a user's agent.	• Variation is faster because the agent can operate visually and locally. • Feedback is immediate because both sides see the canvas.

Cursor	Magic Path	design.md
--------	------------	-----------

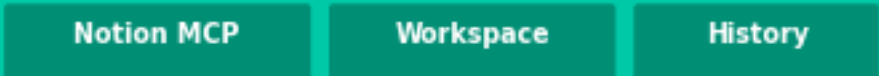
The design tool becomes a shared workspace instead of a separate destination.

[image]

Docs and knowledge work

Notion is already close to agent-native because the workspace is the data model

Why Notion fits	How to design it
• It already stores working context, decisions, and task state. • Notion MCP lets agents read and write workspace content. • The browser gives a shared view for humans and agents.	• Keep source docs, status, and decisions in one place. • Use page history as a memory layer the agent can reuse.
Good pattern	Why it works
• Ask the agent to summarize, update, and link instead of only answer. • Make the output visible in the same workspace.	• Docs turn into live work surfaces instead of static pages. • The tool becomes more useful because the content stays connected.



Documents become agent-native when they are writable, searchable, and reusable.

[image]

Sheets and operations

Structured tables become a control plane when the agent can safely edit them

Why Sheets fits • The data is already structured and auditable. • Many teams already run operations inside spreadsheets. • Formula workflows and row edits are repeatable.	Design rule • Expose row-level actions, not just whole-sheet edits. • Keep lineage visible so users can trust the changes.
What the agent can do • Reconcile rows, update status, and flag anomalies. • Summarize patterns, then write the result back into the sheet.	Why it matters • The agent becomes useful in a control plane the team already uses. • The workflow gains speed without losing auditability.

- Rows
- Audit
- Reconcile

Spreadsheets become agent-native when row actions are explicit and reversible.

[image]

How to pick a wedge

The best opportunity already has a repeated job, a visible surface, and a clear output

Good wedge characteristics	Bad wedge characteristics
• Repeated: the same job happens again and again. • Visible: the work can be reviewed in a shared surface. • Easy to verify: success is obvious to the user.	• One-off: no repeatable pattern to compound. • Hidden: the user cannot see what the agent changed.
Selection rule	Why this matters
• Start where the workflow already exists. • Do not ask users to invent a new habit before the product works.	• The best wedge is already expensive enough to fix. • It is also narrow enough to ship without boiling the ocean.

- Repeated
- Visible
- Auditable

If the job is not repeated, visible, and auditable, it is probably not the wedge.

[image]

Build steps 1-3

Pick the job, define the surface, and build for BYO agent

1-2: job and surface

- Start with one outcome the user and agent should achieve together.
- Choose the shared workspace both sides can edit.
- Define the review flow before you define the UI chrome.

3: BYO agent

- Expose tools so any agent can use the app.
- Do not lock the product to one model or one vendor.

Practical implication

- The product design starts with the shared surface and the tool surface.
- The UI should reveal work, not hide it.

Failure mode

- If the app is just an agent front-end, it is replaceable.
- If the app is a workspace, it becomes sticky.

Job

Surface

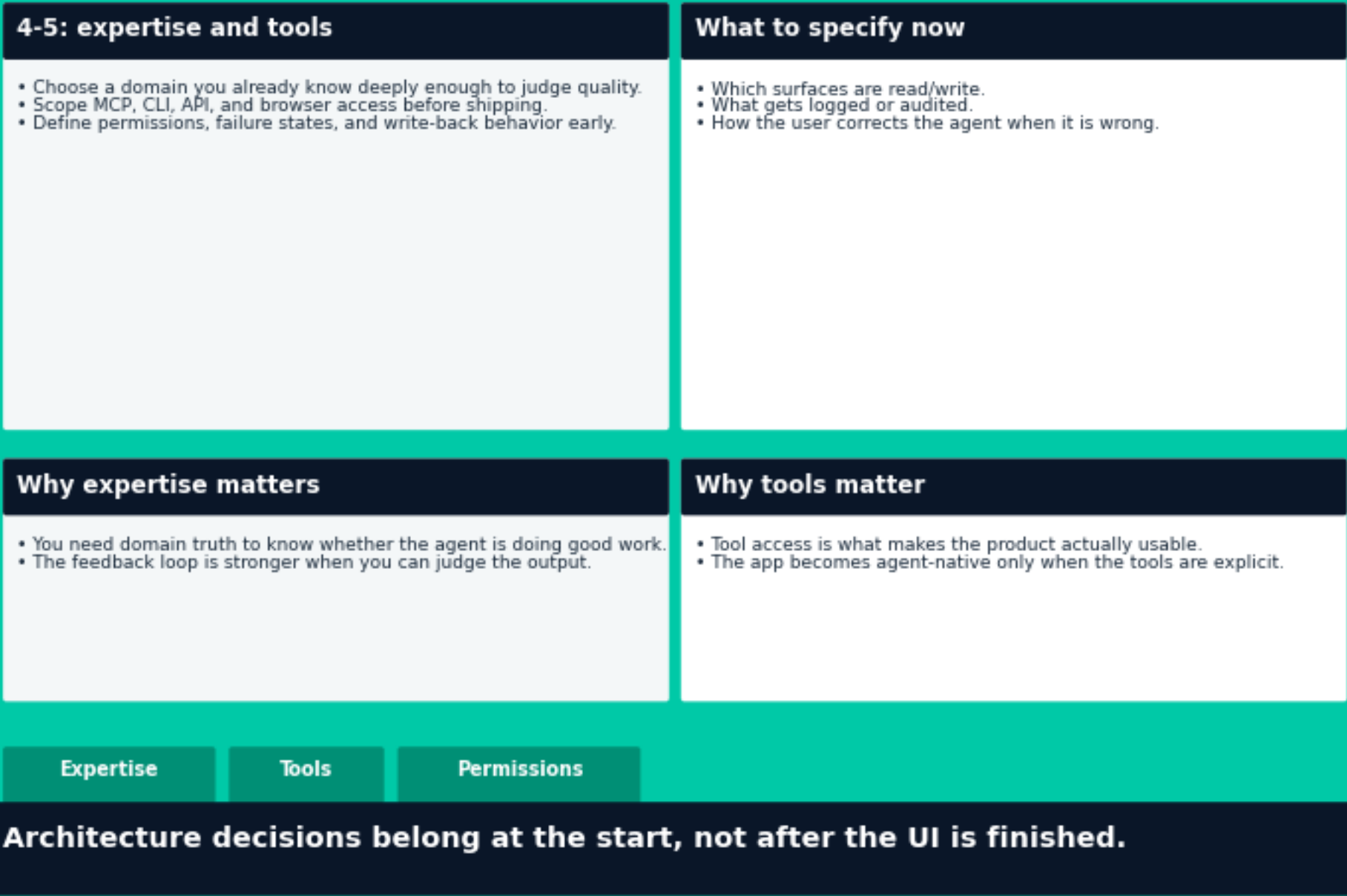
BYO agent

Start with the job and the surface before you start writing the agent loop.

[image]

Build steps 4-5

Build from expertise and plan the tool map on day one



[image]

Anti-patterns

The common ways teams build something that is not actually agent-native

Do not build	Why they fail
• A router agent that only selects fixed functions. • A workflow-shaped black box that hides judgment. • A product that writes state somewhere the user cannot see.	• They reduce the agent to a caller. • They break trust because the work is not legible. • They prevent the user from correcting the outcome.
The fix	Rule of thumb
• Use atomic tools, shared surfaces, and visible state changes. • Keep the collaboration loop in the product.	• If the code already solved the interesting part, the agent is just a caller. • That is not an agent-native product.

- Router
- Black box
- Silent state

Legibility beats cleverness. The user should always know what changed.

[image]

Launch checklist

Use these questions before you commit to the wedge

Go / no-go • Does the user already work here today? • Can the agent reach the tools? • Can both see the same state?	Confidence check • Can the user review and correct the output quickly? • Will the context compound after each run?
If the answer is no • You are asking for behavior change before product value. • The wedge probably needs more architecture work.	If the answer is yes • The product has a real shot at becoming sticky. • The agent can improve without turning into a separate silo.

User works here

Tools reachable

Shared state

Context compounds

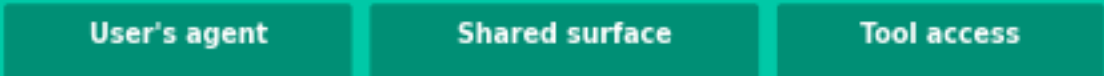
If any answer is no, the idea needs more architecture before it needs more code.

[image]

Bottom line

The category is software redesigned for the user's agent

What to remember	What to do next
• The best new AI opportunity is not another chatbot. • It is software that becomes useful to an external agent.	• Choose one repeated job. • Map the shared surface. • Expose the tools early.
Why this wins	Final rule
• Context compounds instead of resetting. • The user can watch and correct the work. • The app becomes part of the workflow, not a separate destination.	• Build for the user's agent, not just the user.



Agent-native apps compound context, not just clicks.

[image]